

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO ESTADO
DE SÃO PAULO – CAMPUS SÃO PAULO
BACHARELADO EM SISTEMAS DE INFORMAÇÃO**

ANDREY GUSTAVO SEGANTIN DOS SANTOS

ELTON LIMA ARAUJO

JOÃO GUSTAVO DOS SANTOS

ERIK JONATAN MARTINS VIANA

**DESENVOLVIMENTO DE SIMULAÇÃO INTERATIVA DO SISTEMA
SOLAR EM OPENGL**

SÃO PAULO - SP

2026

ANDREY GUSTAVO SEGANTIN DOS SANTOS

ELTON LIMA ARAUJO

JOÃO GUSTAVO DOS SANTOS

ERIK JONATAN MARTINS VIANA

**DESENVOLVIMENTO DE SIMULAÇÃO INTERATIVA DO SISTEMA
SOLAR EM OPENGL**

Trabalho apresentado ao curso de Bacharelado em Sistemas de Informação do Instituto Federal de São Paulo – Campus São Paulo, como requisito para aprovação na disciplina Computação Gráfica desenvolvida pelo Joao Antonio Temochko Andre.

SÃO PAULO - SP

2026

RESUMO

Este trabalho apresenta o desenvolvimento de uma simulação interativa e tridimensional do Sistema Solar, realizada como projeto final da disciplina de Computação Gráfica. A aplicação foi construída utilizando a linguagem de programação C++ e a API gráfica OpenGL, com auxílio da biblioteca GLUT. O sistema implementa uma hierarquia complexa de transformações geométricas para representar as órbitas e rotações dos corpos celestes, utilizando pilhas de matrizes para garantir a fidelidade dos movimentos relativos, como o sistema Terra-Lua. A simulação incorpora técnicas avançadas de renderização, incluindo o mapeamento de texturas UV a partir de arquivos BMP e um modelo de iluminação pontual de Phong, onde o Sol atua como fonte luminosa central. Além da modelagem, o projeto destaca-se pela interatividade, oferecendo uma câmera dinâmica de perseguição e um HUD (Heads-Up Display) informativo desenvolvido em projeção ortográfica 2D. Os resultados obtidos demonstram a eficácia da ferramenta na consolidação de conceitos teóricos de pipeline gráfico e cinemática orbital, resultando em um ambiente virtual educativo, fluido e imersivo.

Palavras-chave: Computação Gráfica. OpenGL. Sistema Solar. Simulação Interativa. C++.

SUMÁRIO

1. INTRODUÇÃO.....	10
2. OBJETIVOS.....	10
2.1 Objetivo Geral.....	10
2.2 Objetivos Específicos.....	11
3. FUNDAMENTAÇÃO TÉCNICA.....	11
3.1 Modelagem e Primitivas Geométricas.....	11
3.2 Carregamento de Texturas e Memória de Vídeo.....	12
3.3 Hierarquia de Transformações e Cinemática Orbital.....	12
3.4 Sistema de Iluminação e Sombreamento (Shading).....	13
3.5 Câmera, Projeção e HUD.....	14
4. DIÁRIO DE DESENVOLVIMENTO.....	14
5. ANÁLISE E DETALHAMENTO DA IMPLEMENTAÇÃO.....	16
5.1 Funções de Inicialização e Recursos.....	16
5.2 Lógica de Renderização e Transformação.....	16
5.3 Interface e Câmera.....	16
5.4 Animação e Interatividade.....	17
5. CONSIDERAÇÕES FINAIS.....	17
6. REFERÊNCIAS.....	18
7. LINK DO PROJETO NO GITHUB.....	18
https://github.com/joaogust/Sistema_Solar_OpenGL	18

1. INTRODUÇÃO

O campo da Computação Gráfica (CG) fundamenta-se na representação e manipulação de dados visuais e geométricos por meio de sistemas computacionais. Este relatório descreve detalhadamente o processo de desenvolvimento de uma simulação tridimensional interativa de um sistema solar simplificado, concebida como o projeto final da disciplina. A aplicação foi construída utilizando a linguagem de programação C++ em conjunto com a API OpenGL (Open Graphics Library) e a biblioteca auxiliar GLUT (OpenGL Utility Toolkit).

O projeto desafia a transição de primitivas geométricas simples para a construção de um ambiente dinâmico e complexo. A simulação não se limita a representar a geometria dos astros, ela integra conceitos fundamentais do pipeline gráfico, como:

- **Cinemática Orbital:** Utilização de funções trigonométricas para o cálculo de translação e rotação sincronizada de corpos celestes.
- **Modelagem Hierárquica:** Organização de transformações geométricas (escalonamento, rotação e translação) por meio de pilhas de matrizes, garantindo que satélites, como a Lua, acompanhem o movimento de seus planetas de referência.
- **Processamento Visual Avançado:** Implementação de iluminação pontual (Point Light) com o Sol atuando como fonte emissora, além do mapeamento de texturas via coordenadas UV para conferir realismo às superfícies.
- **Interatividade e IHM (Interface Homem-Máquina):** Desenvolvimento de uma câmera dinâmica de perseguição e um HUD informativo, transformando a simulação em uma ferramenta pedagógica capaz de exibir dados técnicos como temperatura, diâmetro e períodos orbitais em tempo real.

Dessa forma, o sistema aqui apresentado serve como um consolidado técnico das técnicas de modelagem, iluminação e animação computacional exploradas ao longo do semestre acadêmico.

2. OBJETIVOS

2.1 Objetivo Geral

O objetivo central deste projeto é o desenvolvimento de uma aplicação gráfica tridimensional que simula, de forma interativa e visualmente coerente, a dinâmica orbital e as características físicas dos principais corpos celestes do nosso sistema solar (do Sol a Netuno). O projeto visa consolidar os conhecimentos teóricos de computação gráfica em uma

ferramenta prática que utilize o pipeline de renderização do OpenGL para criar uma experiência educativa e imersiva.

2.2 Objetivos Específicos

Para a consecução do objetivo geral, foram estabelecidas as seguintes metas técnicas:

- **Modelagem Geométrica e Mapeamento:** Construir representações esféricas para o Sol, planetas e satélites naturais, aplicando técnicas de mapeamento UV para a projeção de texturas de alta resolução, garantindo a fidelidade visual das superfícies planetárias.
- **Domínio de Transformações 3D:** Implementar uma hierarquia de matrizes de transformação para gerenciar movimentos complexos, garantindo a correta relação espacial entre a translação orbital e a rotação intrínseca de cada corpo celeste.
- **Simulação de Iluminação Realista:** Configurar o Sol como uma fonte de luz pontual (Point Light) utilizando o modelo de reflexão de Phong, permitindo a visualização de ciclos de dia e noite (sombreamento) nos planetas com base em suas posições relativas.
- **Navegação e Projeção:** Configurar câmeras virtuais em perspectiva e implementar um sistema de projeção ortográfica sobreposta para a criação de uma interface de usuário (HUD) estável.
- **Interatividade e Controle:** Desenvolver um sistema de entrada via teclado que permite ao usuário alternar focos de visualização, controlar o fluxo temporal (velocidade da simulação) e navegar pelo cenário através de ajustes de elevação e zoom.
- **Documentação e Análise Técnica:** Registrar de forma sistemática as decisões de projeto, os desafios lógicos encontrados na programação gráfica e as soluções adotadas pelo grupo durante o ciclo de desenvolvimento.

3. FUNDAMENTAÇÃO TÉCNICA

Nesta seção, detalhamos as escolhas lógicas e as funções da API OpenGL utilizadas para transpor os conceitos astronômicos para o ambiente computacional.

3.1 Modelagem e Primitivas Geométricas

A modelagem de corpos celestes exige uma representação esférica precisa. Para tal, o sistema utiliza a biblioteca GLU (OpenGL Utility Library), especificamente através do objeto

GLUquadric.

- Criação de Quádricas: Em vez de definir manualmente milhares de vértices para cada esfera, utilizamos `gluNewQuadric()`. Este objeto permite configurar propriedades de renderização automáticas, como o estilo de desenho (`GLU_FILL`) para superfícies sólidas e o cálculo automático de normais (`GLU_SMOOTH`), essencial para que a iluminação seja refletida de forma suave e realista.
- A Função `gluSphere`: Cada astro é gerado pela função `gluSphere(quadric, raio, fatias, pilhas)`. O parâmetro "fatias" (longitude) e "pilhas" (latitude) foi configurado para 32x32 nos planetas principais e no Sol, garantindo uma curvatura fluida sem sobrecarregar o processamento da GPU.
- Mapeamento UV: Um requisito técnico fundamental foi o uso de `gluQuadricTexture(quadric, GL_TRUE)`. Esta instrução ordena ao OpenGL que gere automaticamente coordenadas de textura u e v para cada vértice da esfera. Sem isto, as imagens dos planetas não seriam "embrulhadas" corretamente em torno da geometria, resultando em cores sólidas ou distorções.

3.2 Carregamento de Texturas e Memória de Vídeo

O realismo visual é alcançado através do mapeamento de ficheiros de imagem .bmp sobre as esferas. O processo técnico implementado na função `carregarTexturaBMP` segue os seguintes passos:

1. Leitura do Cabeçalho: O sistema abre o ficheiro em modo binário e valida os primeiros 54 bytes (Header BMP), extraindo a largura, altura e a posição dos dados de cor.
2. Conversão de Formato (BGR para RGB): Como o formato Windows BMP armazena os canais de cor na ordem Blue-Green-Red (BGR), o código utiliza a constante `GL_BGR` na função `glTexImage2D` para garantir que as cores dos planetas (como o azul da Terra) não sejam exibidas invertidas.
3. Configuração de Filtros: Foram aplicados filtros de magnificação e minificação `GL_LINEAR`. Isto garante que, ao aproximar a câmara de um planeta (Zoom), o OpenGL realize uma interpolação linear entre os pixéis da textura, evitando o efeito de "pixelização" e mantendo a suavidade visual.

3.3 Hierarquia de Transformações e Cinemática Orbital

A simulação de múltiplos corpos em movimento exige uma gestão rigorosa das

matrizes de transformação. O projeto utiliza o conceito de Pilha de Matrizes (`glPushMatrix` e `glPopMatrix`) para criar uma hierarquia de movimentos.

- **Ordem das Transformações:** Para cada planeta, a sequência de comandos é fundamental para o resultado visual:
 1. `glRotatef(anguloOrbita, 0, 1, 0)`: Aplica-se a rotação orbital antes da translação. Como o objeto ainda está na origem (0,0,0), rotacionar o sistema de coordenadas e depois transladar cria o movimento circular de translação ao redor do Sol.
 2. `glTranslatef(distancia, 0, 0)`: Afasta o planeta do centro solar conforme sua distância orbital específica.
 3. `glRotatef(anguloRotacao, 0, 1, 0)`: Aplica-se a rotação intrínseca (rotação sobre o próprio eixo) após o planeta estar em sua posição orbital.
- **Sistemas Hierárquicos (O caso Terra-Lua):** A Lua representa um desafio técnico de modelagem hierárquica. Para que ela acompanhe a Terra em sua órbita, as transformações da Lua são realizadas dentro do escopo da matriz da Terra. Ou seja, a Lua "herda" a posição da Terra e adiciona seu próprio deslocamento e rotação relativa, demonstrando o uso de transformações compostas.

3.4 Sistema de Iluminação e Sombreamento (Shading)

Para conferir tridimensionalidade e realismo à cena, foi implementado um modelo de iluminação baseado no modelo de reflexão de Phong.

- **Fonte de Luz Pontual (Point Light):** O Sol é configurado como a única fonte de luz ativa (`GL_LIGHT0`). Sua posição é fixada na origem (0,0,0) com o parâmetro `w = 1.0`, o que define a luz como pontual (emitindo raios em todas as direções), simulando o comportamento físico de uma estrela.
- **Propriedades de Material:** Os planetas possuem propriedades difusas e especulares configuradas para reagir à luz solar. Isso permite a visualização do fenômeno de "fases", onde apenas a face voltada para o Sol é iluminada, enquanto a face oposta permanece em sombra.
- **Emissão do Sol:** Tecnicamente, no OpenGL, uma fonte de luz não "brilha" visualmente por si só. Para resolver isso, o Sol é desenhado com a iluminação desativada (`glDisable(GL_LIGHTING)`) e uma cor branca/amarela pura associada à

sua textura. Isso garante que o astro central seja percebido como o ponto mais brilhante do sistema, sem sofrer influência de sombras externas.

3.5 Câmera, Projeção e HUD

A experiência do usuário é mediada por um sistema complexo de visualização que alterna entre modos de projeção.

- **Projeção em Perspectiva:** Utiliza-se `gluPerspective` para criar a ilusão de profundidade, onde objetos distantes parecem menores. O campo de visão (*Field of View*) foi ajustado para 45 graus, proporcionando uma visão ampla do sistema sem distorções excessivas nas bordas.
- **Câmera Dinâmica:** A função `gluLookAt` é atualizada em tempo real. Quando um planeta é selecionado (teclas 0-8), o ponto de destino (*look-at*) é recalculado para as coordenadas X e Z atuais do planeta, garantindo que o foco da câmera acompanhe o astro em sua órbita.
- **Interface 2D (Overlay):** O HUD é renderizado em uma camada superior utilizando Projeção Ortográfica (`gluOrtho2D`). Esta técnica "achata" a renderização para que o texto e os painéis de informação permaneçam fixos em relação à tela do monitor, independentemente das manobras da câmera 3D ao fundo.

4. DIÁRIO DE DESENVOLVIMENTO

Nesta seção, registramos a evolução cronológica do projeto e as principais decisões técnicas tomadas pelo grupo durante o ciclo de desenvolvimento.

4.1.

- **Reunião 1 (Definição de Escopo e Arquitetura):** O grupo estabeleceu que a expansão para o sistema solar completo agregaria maior valor técnico. Decidiu-se pela utilização do formato BMP (Bitmap) para texturas, visando implementar um carregador de baixo nível próprio, garantindo total controle sobre a manipulação de bytes e evitando dependências externas.
- **Reunião 2 (Modelagem Hierárquica):** Foco na implementação das translações. O desafio surgiu no posicionamento da Lua em relação à Terra. A solução foi o estudo aprofundado da pilha de matrizes, garantindo que o satélite herdasse as transformações do planeta pai.
- **Reunião 3 (Pipeline de Texturização):** Com o carregador funcional, notou-se

que o padrão BMP armazena dados em BGR, o que exigiu ajustes no parâmetro de formato da função `glTexImage2D`. Configuramos também a iluminação pontual no centro do Sol.

- Reunião 4 (IHM e Gerenciamento de Estado): Criação do HUD com estética Sci-Fi. A complexidade residiu na transição entre os contextos 3D e 2D no mesmo frame, exigindo um rigoroso sistema de reset de estados do OpenGL.

4.2. Dificuldades Encontradas e Soluções Técnicas

Durante o desenvolvimento, o grupo enfrentou obstáculos que exigiram pesquisa e experimentação. Abaixo, detalhamos os principais desafios:

Conflito de Estados no HUD: Ao implementar as informações textuais, o texto inicialmente aparecia com as texturas dos planetas ou desaparecia atrás de objetos 3D.

- Solução: Implementamos um par de funções (`iniciaHUD` e `finalizaHUD`) que desativa o teste de profundidade (`GL_DEPTH_TEST`) e o mapeamento de textura antes de renderizar o texto em projeção ortográfica.

Inversão de Cores nas Texturas: As texturas carregadas via BMP apresentavam tons de azul onde deveria haver vermelho (e vice-versa).

- Solução: Identificamos que o formato BMP é nativamente Little-Endian (BGR). Em vez de reordenar os bytes manualmente via CPU, utilizamos a flag `GL_BGR` do OpenGL para que a GPU fizesse a interpretação correta durante a carga da textura.

Iluminação de Planetas Distantes: Devido à queda natural da intensidade da luz pontual, planetas como Urano e Netuno ficavam excessivamente escuros.

- Solução: Ajustamos o modelo de iluminação global e a luz ambiente (`GL_LIGHT_MODEL_AMBIENT`) para garantir visibilidade mínima constante, sem comprometer o efeito de sombreamento (fases) nos planetas rochosos.

Sincronismo da Câmera Dinâmica: A câmera apresentava "trepidação" ao seguir planetas em alta velocidade.

- Solução: Refinamos o cálculo trigonométrico da posição do alvo no `display()`, garantindo que o vetor de visão da câmera fosse atualizado no mesmo passo de simulação que a translação orbital.

5. ANÁLISE E DETALHAMENTO DA IMPLEMENTAÇÃO

A implementação foi estruturada de forma modular para facilitar a manutenção e a legibilidade. Abaixo, detalhamos as principais funções que compõem o sistema.

5.1 Funções de Inicialização e Recursos

- `inicializa()`: Responsável por configurar o estado inicial do OpenGL. Ativa o teste de profundidade (`GL_DEPTH_TEST`), a iluminação global e a normalização de vetores. É nesta função que o objeto quadric é criado e as texturas são carregadas para a memória de vídeo (VRAM). Também gera o campo estático de 2000 estrelas utilizando coordenadas esféricas aleatórias.
- `carregarTexturaBMP(const char* caminho)`: Realiza a leitura binária de arquivos de imagem. Ela interpreta o cabeçalho de 54 bytes do formato BMP para extrair dimensões e envia os dados para a GPU via `glTexImage2D`. Um detalhe técnico importante é o uso do formato `GL_BGR`, necessário porque o padrão BMP armazena cores na ordem Azul-Verde-Vermelho.

5.2 Lógica de Renderização e Transformação

- `desenhaPlaneta(...)`: É a função mais complexa do sistema. Ela gerencia a hierarquia de transformações de cada astro. Utiliza `glPushMatrix()` para isolar as transformações de um planeta das demais. A sequência lógica é:
 1. Desenho da órbita (`GL_LINE_LOOP`);
 2. Rotação orbital;
 3. Translação até a distância do Sol;
 4. Rotação em torno do próprio eixo;
 5. Aplicação da textura e desenho da esfera. Casos especiais, como os anéis de Saturno e a Lua, são desenhados dentro desta função, herdando a posição do planeta pai.
- `desenhaOrbita(float distancia)`: Utiliza um laço de repetição e funções trigonométricas (`cosf` e `sinf`) para desenhar um anel de 10.000 segmentos lineares, servindo como guia visual para a trajetória planetária.
- `configuraLuzSol()`: Define a fonte de luz pontual. Configura a `GL_LIGHT0` na posição central (0,0,0) com o componente `w=1.0`. Isso garante que a luz emane do Sol para todos os lados, sombreando as faces ocultas dos planetas.

5.3 Interface e Câmera

- `iniciaHUD()` e `finalizaHUD()`: Estas funções realizam a "troca de contexto". Elas salvam a matriz de projeção em perspectiva e ativam uma matriz ortográfica 2D (`gluOrtho2D`). Isso permite desenhar elementos estáticos na tela, como textos e painéis, sem que eles se movam com a câmera 3D.
- `desenhaHUD()`: Organiza o layout visual do terminal de informações. Utiliza `GL_QUADS` para o fundo translúcido e `GL_LINE_LOOP` para a moldura, além de extrair os dados da estrutura `InfoPlaneta` para exibição textual.
- `display()`: O "coração" do loop de renderização. Ela limpa os buffers de cor e profundidade, calcula a posição da câmera dinâmica usando `gluLookAt` (baseado no planeta selecionado) e chama as funções de desenho de todos os astros do sistema.

5.4 Animação e Interatividade

- `atualizaAnimacao(int valor)`: Função de *callback* que incrementa as variáveis de ângulo de órbita e rotação. Ela é chamada aproximadamente 60 vezes por segundo via `glutTimerFunc`, garantindo uma animação fluida. Inclui lógica para evitar o *overflow* das variáveis de ângulo, resetando-as ao atingirem 360 graus.
- `teclado(unsigned char key, int x, int y)`: Captura a entrada do usuário. Mapeia as teclas numéricas (0-8) para alterar a variável `planetaSelecionado`, as teclas W/S para elevação da câmera, +/- para zoom e A/D para manipular o fator de velocidade da simulação.

5. CONSIDERAÇÕES FINAIS

O desenvolvimento deste projeto permitiu a aplicação prática e integrada de conceitos fundamentais da disciplina de Computação Gráfica. A transposição de fórmulas matemáticas de cinemática orbital e ótica para um ambiente tridimensional em tempo real demonstrou ser um desafio técnico recompensador, consolidando o aprendizado sobre o pipeline de renderização do OpenGL.

A implementação atingiu plenamente todos os requisitos solicitados, superando as expectativas iniciais ao entregar uma interface de usuário (HUD) interativa e um sistema de câmeras dinâmicas de perseguição. A experiência revelou que a qualidade de uma simulação 3D depende criticamente de três fatores:

1. A precisão na hierarquia de transformações: Onde a ordem de operações de matrizes define o sucesso da modelagem de sistemas complexos como a

relação Terra-Lua.

2. A gestão rigorosa de estados: O controle sobre luzes, texturas e profundidade para evitar artefatos visuais.

3. A experiência do usuário: A capacidade de navegar e extrair informações úteis da simulação de forma intuitiva.

6. REFERÊNCIAS

KHRONOS GROUP. **OpenGL Documentation**. Disponível em: <https://www.opengl.org/documentation/>. Acesso em: 14 mai. 2026.

7. LINK DO PROJETO NO GITHUB

https://github.com/joaogust/Sistema_Solar_OpenGL